



# CSE 3060

## Database System II

---



Lecture #3



Data Definition Language (DDL)



Dr Khine Zar Thwe

Lecturer

Faculty of Information Science

Myanmar Institute of Information Technology





# What will be learned today?

- ✓ Data Definition Language (DDL)
- ✓ DDL – CREATE Statement
- ✓ DDL – DROP Statement
- ✓ DDL – TRUNCATE Statement
- ✓ DDL – RENAME Statement
- ✓ DDL – ALTER Statement
- ✓ Viewing Database Structures

# Data Definition Language (DDL)

- A set SQL commands
  - Used **to define the structures of a database**
  - Used **to create and modify** the structure of database objects
- Common DDL commands include:

|          |                                                  |
|----------|--------------------------------------------------|
| CREATE   | • To create the database schema or its objects   |
| DROP     | • To delete database schema or its objects       |
| ALTER    | • To alter the structure of the database objects |
| RENAME   | • To rename an existing object in the database   |
| TRUNCATE | • To delete all data from table                  |

# Data Definition Language - **CREATE** Statement



- The **CREATE** command is a main SQL command for data definition statement
- Used to create
  - Schema (Database)
  - Table (Relation)
  - Domain
  - View
  - Assertion
  - Stored Procedure
  - Stored Function
  - Trigger

# SQL Schema



- Database schema is the logical structure of a database.
- A SQL schema
  - Identified by a schema name
  - Includes an authorization identifier to indicate the user or account who owns the schema, as well as descriptors for each element in the schema
- Schema elements
  - Tables
  - Constraints
  - Domains
  - Authorization grants

# Data Definition Language - **CREATE** Statement

## Creating a Schema

- Syntax: Create a schema

database name



**CREATE SCHEMA** < schema\_name >;

**CREATE DATABASE** < database\_name >;

- Example

**CREATE SCHEMA** COMPANY;

**CREATE SCHEMA** UNIVERSIY AUTORIZATION 'John';

# Using a Schema

- Enabling a database as the default database

- Syntax

**USE** < database\_name >;

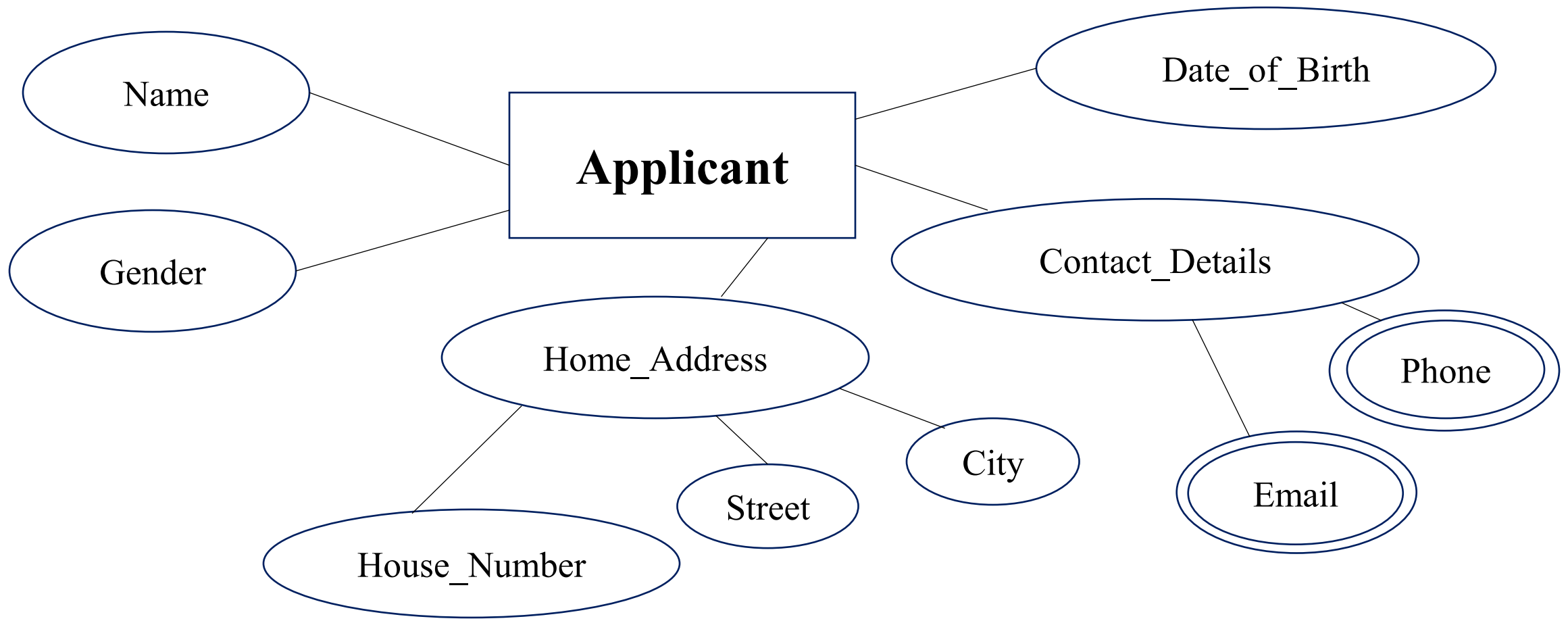
- Example

**USE** COMPANY;

**USE** UNIVERSITY;

- Subsequent statements will be executed for the currently open database

# Example: Entity Applicant





# Data Definition Language - CREATE Statement

## Creating a Table

➤ CREATE command can also be used to create tables.

```
CREATE TABLE < TABLE_NAME > or < SCHEMA_NAME.TABLE_NAME >
```

```
(
```

```
    A1 name          A1 DATA TYPE          [A1 constraints],
```

```
    A2 name          A2 DATA TYPE          [A2 constraints],
```

```
    ...
```

```
    An name          An DATA TYPE          [An constraints],
```

```
PRIMARY KEY (attribute-name),
```

```
FOREIGN KEY (attribute-name) REFERENCES TABLE_NAME (primary key attribute-name),
```

```
UNIQUE (attribute-name)
```

```
);
```

# CREATE TABLE Statement



## Table name

- Name to create Table

## Attributes

- Name, Data type and constraints (if any) of each attribute

## Constraints

- (Key, Entity or Referential Integrity Constraints)

# Example - CREATE TABLE Statement

➤ Create a Customer Table in alternative ways:

```
CREATE TABLE Customer
```

```
(  
    CustomerID varchar(10),  
    CustomerName varchar(20) NOT NULL,  
    ContactNo varchar(11) UNIQUE,  
    Address    varchar(50),  
    PRIMARY KEY (CustomerID)  
);
```

Primary key constraint is defined at table level.

```
CREATE TABLE Customer
```

```
(  
    CustomerID varchar(10) PRIMARY KEY,  
    CustomerName varchar(20) NOT NULL,  
    ContactNo varchar(11) UNIQUE,  
    Address    varchar(50),  
);
```

Primary key constraint is defined at column level.

# Difference between Primary Key and Unique Key



- Purpose: Primary Key is used to uniquely identify a row but a unique key is used to prevent duplicate values in a column.
- Existence : A table can have only one primary key but it can have multiple unique keys.

**Student**

| Roll_no | Name    | Class | Phone_no   | Registration_no |
|---------|---------|-------|------------|-----------------|
| 1       | Andrew  | 5     | 9854672256 | 895             |
| 2       | Andrew  | 6     | 9955512456 | 564             |
| 3       | Augusto | 5     |            | 567             |

Primary  
Key

Unique  
Key

Unique  
Key

# Data Definition Language - DROP Statement

## Drop a Schema

- To remove a schema
- Syntax

**DROP SCHEMA** < schema\_name >;

**DROP DATABASE** < database\_name >;

- Example

**DROP SCHEMA** UNIVERSITY;

# Data Definition Language - DROP Statement

## Drop a Table

- To remove a table from the database.
- Syntax

**DROP TABLE** < table\_name >;

- Example

**DROP TABLE** Applicant;

### **Important**

Be extremely careful when using DROP statement, as there is no undo possible

# Data Definition Language - TRUNCATE Statement



- **TRUNCATE** command

- To delete all data from a table, not the table.

- Syntax

**TRUNCATE TABLE** < table\_name >;

- Example

**TRUNCATE TABLE** Applicant;

**Important**

Be extremely careful when using TRUNCATE statement, as there is no undo possible

# Data Definition Language - RENAME Statement

- **RENAME** command

- To set a new name to a table.

- Syntax

**RENAME TABLE** < old\_table\_name > **TO** < new\_table\_name >;

- Example

**RENAME TABLE** Applicant **TO** Company\_Applicant;



# Data Definition Language - ALTER Statement

- **ALTER** command (altering the table structure)
  - To add, modify, rename and delete columns in a table
  - To add and drop constraints on a table
- Syntax

**ALTER TABLE** < table\_name >

< action >;

# Data Definition Language - ALTER Statement

## Adding Columns to a Table

### ➤ Add a new Column

```
ALTER TABLE < table_name >
```

```
ADD COLUMN column_name datatype [FIRST | AFTER existing column];
```

### ➤ Add multiple new Columns

```
ALTER TABLE < table_name >
```

```
ADD COLUMN (column_name1 datatype1,  
            column_name2 datatype2  
            column_name3 datatype3);
```

# Data Definition Language - ALTER Statement

## Deleting a Column from a Table

- Delete an existing column

**ALTER TABLE**    < table\_name >

**DROP COLUMN**   column\_name;

# Data Definition Language - ALTER Statement

## Modifying an existing Column of a Table

- Modify the datatype of an existing column

**ALTER TABLE** < table\_name >

**MODIFY COLUMN** column\_name datatype;

- Change the name of an existing column

**ALTER TABLE** < table\_name >

**CHANGE COLUMN** old\_column\_name new\_column\_name datatype;

# Data Definition Language - ALTER Statement

## Adding, Deleting, and Modifying constraints in a Table

- Add a constraint to an existing column

**ALTER TABLE** < table\_name >

**MODIFY COLUMN** column\_name datatype **NOT NULL**;

- Add PRIMARY KEY to an existing column

**ALTER TABLE** < table\_name >

**ADD PRIMARY KEY** (column\_name);

- Delete PRIMARY KEY from the table

**ALTER TABLE** < table\_name >

**DROP PRIMARY KEY**

# View Database Structures



- Open database

**USE** <database\_name>;

**USE** Temporary\_DB;

- Display the selected the database

**SELECT DATABASE();**

- Display the tables in the database

**SHOW TABLES;**

- Display the table structure

**DESCRIBE** <table\_name>;

**DESCRIBE** Company\_Applicant;

**SHOW COLUMNS FROM** <table\_name>;

**SHOW COLUMNS FROM** Company\_Applicant;

- Display the create table command

**SHOW CREATE TABLE** <table\_name>;

**SHOW CREATE TABLE** Company\_Applicant;

- Display the create schema command

**SHOW CREATE SCHEMA** <database\_name>;

**SHOW CREATE SCHEMA** Temporary\_DB;

**Next Lecture**



**Practice Session 1**